

Naro Fashion - Production Deployment Plan

Context

The Naro Fashion monorepo (3 apps + shared packages) needs to go from localhost development to a production deployment accessible at **narofashion.co.tz**. This plan covers domain registration, hosting selection, server setup, deployment automation, and security hardening.

Part 1: Domain Name — narofashion.co.tz

.co.tz Domain Registration

`.co.tz` domains are managed by the **Tanzania Network Information Centre (tzNIC)** — <https://tznictz.or.tz>

Option A — Register directly via tzNIC-accredited registrars (recommended for .co.tz):

- **TZ Domains** (tzdomain.com) — ~\$25-35/year for .co.tz
- **Angani** (angani.co) — East Africa focused, ~\$30/year
- **Habari Node** (habarinode.co.tz) — Tanzania local registrar

Option B — International registrars that support .co.tz:

- **Namecheap** — check availability, usually \$30-50/year
- **Gandi.net** — supports .tz TLDs

Recommendation: Use a **Tanzania-based registrar** (TZ Domains or Habari Node) for the best price and local support. Budget: ~\$25-35/year.

DNS Configuration (after domain is registered)

A	narofashion.co.tz	→ <server-ip>
A	www.narofashion.co.tz	→ <server-ip>
A	admin.narofashion.co.tz	→ <server-ip>
A	api.narofashion.co.tz	→ <server-ip>

Part 2: Hosting — Server Selection

Recommended: Hetzner Cloud (Best price-to-performance)

Why Hetzner:

- Cheapest reliable VPS globally, well-regarded by developers
- Data center in **Falkenstein/Nuremberg, Germany** (decent latency to East Africa ~120ms)
- No bandwidth caps (20TB included)

Recommended Plan: CX32 (or CPX31)

Spec	Value
vCPUs	4
RAM	8 GB

Storage	80 GB NVMe SSD
Price	\$7.50/month (€6.90)

Step-by-Step: Creating Your Hetzner Server

1. Create a Hetzner Account

1. Go to <https://accounts.hetzner.com/signUp>
2. Register with your email address
3. Verify your email and complete identity verification (may require ID upload)
4. Add a payment method (credit card or PayPal)

2. Create a Cloud Project

1. Go to <https://console.hetzner.cloud>
2. Click "+ New Project" → name it `naro-fashion`
3. Enter the project

3. Add Your SSH Key (do this first!)

On your local machine, generate an SSH key if you don't have one:

```
ssh-keygen -t ed25519 -C "your-email@example.com"
cat ~/.ssh/id_ed25519.pub # copy this output
```

In Hetzner Console:

1. Go to **Security** → **SSH Keys** → **Add SSH Key**
2. Paste your public key and name it (e.g., `my-laptop`)

4. Create the Server

1. Click "+ Create Server"
2. **Location:** Falkenstein (FSN1) — closest to East Africa
3. **Image:** Ubuntu 24.04 LTS
4. **Type:** Shared vCPU → **CX32** (4 vCPU, 8GB RAM, 80GB SSD — ~€6.90/mo)
5. **Networking:** Leave default (public IPv4 + IPv6)
6. **SSH Key:** Select the key you added in step 3
7. **Name:** `naro-fashion-prod`
8. Click "**Create & Buy Now**"

5. Note Your Server IP

After creation, copy the **IPv4 address** shown (e.g., `65.108.x.x`). You'll need this for:

- DNS configuration (Part 1)
- SSH access: `ssh root@65.108.x.x`
- GitHub Actions secrets (Part 4)

6. First SSH Connection

```
ssh root@<your-server-ip>
# You should get in without a password (SSH key auth)
```

This handles all 3 apps + PostgreSQL comfortably for a new e-commerce site.

Alternatives considered:

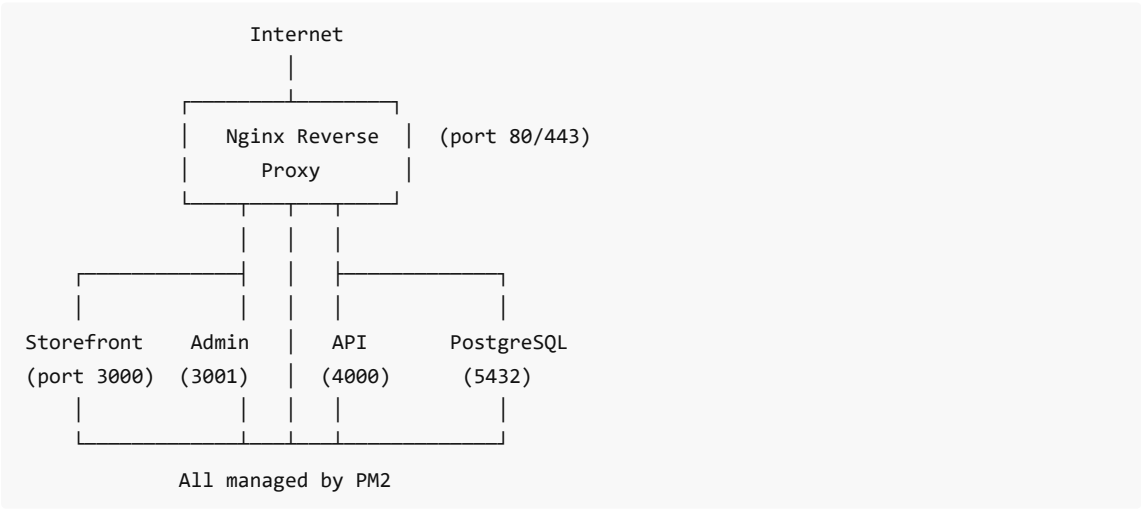
Provider	Plan	Price/mo	Notes
Hetzner CX32	4 vCPU, 8GB RAM	~\$7.50	Best value, recommended
DigitalOcean	4 vCPU, 8GB RAM	~\$48	More expensive, good docs
Contabo	6 vCPU, 16GB RAM	~\$7	Cheap but inconsistent performance
Vultr	4 vCPU, 8GB RAM	~\$48	Good but pricey
Railway/Render	PaaS	~\$20-50+	Easier but costs add up with 3 apps + DB

Total Monthly Cost Estimate

Item	Cost
Hetzner CX32 VPS	~\$7.50/mo
Domain (.co.tz)	~\$2.50/mo (annual)
SSL (Let's Encrypt)	Free
Email (optional, Zoho free tier)	Free
Total	~\$10/month

Part 3: Server Setup & Deployment Architecture

Architecture Overview



Domain Routing

URL	Routes to
narofashion.co.tz / www.narofashion.co.tz	Storefront (port 3000)

admin.narofashion.co.tz	Admin dashboard (port 3001)
api.narofashion.co.tz	NestJS API (port 4000)

Step-by-Step Server Setup

Step 1: Initial Server Setup (Ubuntu 24.04 LTS)

```
# Update system
sudo apt update && sudo apt upgrade -y

# Create deploy user
sudo adduser deploy
sudo usermod -aG sudo deploy

# Set up SSH key auth (disable password login after)
sudo nano /etc/ssh/sshd_config
# → PasswordAuthentication no
# → PermitRootLogin no
sudo systemctl restart sshd

# Firewall
sudo ufw allow OpenSSH
sudo ufw allow 80
sudo ufw allow 443
sudo ufw enable
```

Step 2: Install Dependencies

```
# Node.js 22 LTS (via NodeSource)
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs

# pnpm
corepack enable
corepack prepare pnpm@10.12.1 --activate

# PM2 (process manager)
npm install -g pm2

# Nginx
sudo apt install -y nginx

# PostgreSQL 17
sudo apt install -y postgresql-17 postgresql-contrib-17
```

Step 3: PostgreSQL Setup

```
sudo -u postgres psql
```

```
CREATE USER naro_admin WITH PASSWORD '<strong-random-password>';
CREATE DATABASE naro_fashion OWNER naro_admin;
GRANT ALL PRIVILEGES ON DATABASE naro_fashion TO naro_admin;
\q
```

Step 4: Clone & Build the Project

```
# As deploy user
cd /home/deploy
git clone https://github.com/rjurio/naro-fashion.git
cd naro-fashion

# Install dependencies
pnpm install --frozen-lockfile

# Create production .env
cp .env .env.production
# Edit with production values (see Environment Variables section below)

# Generate Prisma client & push schema
cd packages/database
pnpm db:generate
pnpm db:push
cd ../../

# Build all apps
pnpm build
```

Step 5: Production Environment Variables

Create `/home/deploy/naro-fashion/.env` :

```
# Database
DATABASE_URL="postgresql://naro_admin:<STRONG_PASSWORD>@localhost:5432/naro_fashion?
schema=public"

# URLs (production)
STOREFRONT_URL="https://narofashion.co.tz"
ADMIN_URL="https://admin.narofashion.co.tz"
API_URL="https://api.narofashion.co.tz"
NEXT_PUBLIC_API_URL="https://api.narofashion.co.tz/api/v1"

# JWT (generate strong random secrets!)
JWT_SECRET="<generate-with: openssl rand -base64 64>"
JWT_REFRESH_SECRET="<generate-with: openssl rand -base64 64>"
JWT_EXPIRATION="15m"
JWT_REFRESH_EXPIRATION="7d"
```

```
# Node
NODE_ENV=production
```

Also create `apps/storefront/.env.local` and `apps/admin/.env.local` :

```
NEXT_PUBLIC_API_URL=https://api.narofashion.co.tz/api/v1
```

Step 6: PM2 Process Management

Create `/home/deploy/naro-fashion/ecosystem.config.js` :

```
module.exports = {
  apps: [
    {
      name: 'storefront',
      cwd: './apps/storefront',
      script: 'node_modules/.bin/next',
      args: 'start -p 3000',
      env: { NODE_ENV: 'production', PORT: 3000 }
    },
    {
      name: 'admin',
      cwd: './apps/admin',
      script: 'node_modules/.bin/next',
      args: 'start -p 3001',
      env: { NODE_ENV: 'production', PORT: 3001 }
    },
    {
      name: 'api',
      cwd: './apps/api',
      script: 'dist/main.js',
      env: { NODE_ENV: 'production', PORT: 4000 }
    }
  ]
};
```

```
pm2 start ecosystem.config.js
pm2 save
pm2 startup # auto-start on reboot
```

Step 7: Nginx Reverse Proxy

Create `/etc/nginx/sites-available/narofashion` :

```
# Storefront
server {
    listen 80;
    server_name narofashion.co.tz www.narofashion.co.tz;

    location / {
```

```

        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }
}

# Admin
server {
    listen 80;
    server_name admin.narofashion.co.tz;

    location / {
        proxy_pass http://127.0.0.1:3001;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }
}

# API
server {
    listen 80;
    server_name api.narofashion.co.tz;

    client_max_body_size 10M; # for file uploads

    location / {
        proxy_pass http://127.0.0.1:4000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }
}

```

```
sudo ln -s /etc/nginx/sites-available/narofashion /etc/nginx/sites-enabled/  
sudo rm /etc/nginx/sites-enabled/default  
sudo nginx -t  
sudo systemctl reload nginx
```

Step 8: SSL with Let's Encrypt (Free HTTPS)

```
sudo apt install -y certbot python3-certbot-nginx  
sudo certbot --nginx -d narofashion.co.tz -d www.narofashion.co.tz -d  
admin.narofashion.co.tz -d api.narofashion.co.tz  
  
# Auto-renewal is set up automatically by certbot  
sudo certbot renew --dry-run # test
```

Part 4: CI/CD Pipeline (GitHub Actions)

How It Works

Push to `master` → GitHub Actions builds & tests → SSH into server → deploy automatically.

Prerequisites on Server

1. Generate an SSH key pair for GitHub Actions:

```
ssh-keygen -t ed25519 -C "github-actions" -f ~/.ssh/github_actions  
cat ~/.ssh/github_actions.pub >> ~/.ssh/authorized_keys
```

2. Copy the **private key** content — you'll add it as a GitHub secret.

GitHub Repository Secrets

Go to **GitHub** → **Settings** → **Secrets and variables** → **Actions** and add:

Secret Name	Value
SERVER_HOST	Your Hetzner server IP (e.g., 65.108.x.x)
SERVER_USER	deploy
SERVER_SSH_KEY	Contents of ~/.ssh/github_actions (private key)
DATABASE_URL	postgresql://naro_admin:<password>@localhost:5432/naro_fashion?schema=public

GitHub Actions Workflow

Create `.github/workflows/deploy.yml` :

```
name: Build & Deploy  
  
on:  
  push:
```

```
branches: [master]
```

```
jobs:
```

```
  build-and-test:
```

```
    name: Build & Validate
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

- uses: actions/checkout@v4
- uses: pnpm/action-setup@v4
 with:
 version: 10.12.1
- uses: actions/setup-node@v4
 with:
 node-version: 22
 cache: 'pnpm'
- name: Install dependencies
 run: pnpm install --frozen-lockfile
- name: Generate Prisma client
 run: cd packages/database && pnpm db:generate
- name: Lint
 run: pnpm lint
- name: Build all apps
 run: pnpm build

```
  deploy:
```

```
    name: Deploy to Production
```

```
    needs: build-and-test
```

```
    runs-on: ubuntu-latest
```

```
    if: github.ref == 'refs/heads/master'
```

```
    steps:
```

- name: Deploy via SSH
 uses: appleboy/ssh-action@v1
 with:
 host: \${ secrets.SERVER_HOST }
 username: \${ secrets.SERVER_USER }
 key: \${ secrets.SERVER_SSH_KEY }
 script: |
 cd /home/deploy/naro-fashion
 git pull origin master
 pnpm install --frozen-lockfile
 cd packages/database && pnpm db:generate && pnpm db:push && cd ../..
 pnpm build
 pm2 restart all
 echo "Deployment successful at \$(date)"

Workflow Summary

Push to master → Build & Lint → Deploy to Production

Manual Deploy Script (backup/fallback)

Also create `/home/deploy/naro-fashion/deploy.sh` for manual deployments:

```
#!/bin/bash
set -e

echo "Pulling latest code..."
git pull origin master

echo "Installing dependencies..."
pnpm install --frozen-lockfile

echo "Generating Prisma client..."
cd packages/database && pnpm db:generate && cd ../..

echo "Running migrations..."
cd packages/database && pnpm db:push && cd ../..

echo "Building all apps..."
pnpm build

echo "Restarting services..."
pm2 restart all

echo "Deployment complete!"
```

```
chmod +x deploy.sh
# To deploy manually: ./deploy.sh
```

Part 5: Security Hardening

Server Security

- ☒ SSH key-only auth (no passwords)
- ☒ UFW firewall (only 22, 80, 443)
- ☒ Non-root deploy user
- ☐ Install **fail2ban**: `sudo apt install fail2ban`
- ☐ Automatic security updates: `sudo apt install unattended-upgrades`

Application Security

- ☒ Helmet middleware (already in API)
- ☒ CORS configured (already in API)
- ☒ Rate limiting via `@nestjs/throttler` (already in API)

- ☐ Change all default passwords and JWT secrets
- ☐ Set `secure: true` on cookies in production
- ☐ Restrict admin subdomain access by IP (optional, via Nginx `allow/deny`)

Database Security

- ☐ Use a strong random password (not Admin123)
- ☐ PostgreSQL only listens on localhost (default)
- ☐ Regular automated backups (see below)

Database Backups

```
# Daily backup cron job
crontab -e
# Add:
0 2 * * * pg_dump -U naro_admin naro_fashion | gzip >
/home/deploy/backups/naro_fashion_$(date +%Y%m%d).sql.gz
```

Part 6: Monitoring & Logging

Uptime Monitoring — UptimeRobot (Free)

1. Sign up at <https://uptimerobot.com> (free tier: 50 monitors, 5-min checks)
2. Add 3 HTTP(S) monitors:
 - `https://narofashion.co.tz` — storefront
 - `https://admin.narofashion.co.tz` — admin
 - `https://api.narofashion.co.tz/api/v1` — API health
3. Set alert contacts (email + optional Telegram/Slack)
4. You'll get instant email alerts when any service goes down

PM2 Monitoring & Logs

PM2 provides built-in logging and monitoring:

```
# View all logs (real-time)
pm2 logs

# View logs for specific app
pm2 logs api
pm2 logs storefront

# Monitor CPU/memory in real-time
pm2 monit

# Log rotation (prevent disk fill)
pm2 install pm2-logrotate
pm2 set pm2-logrotate:max_size 10M
pm2 set pm2-logrotate:retain 7
pm2 set pm2-logrotate:compress true
```

Log files are stored at:

- `/home/deploy/.pm2/logs/api-out.log` — API stdout
- `/home/deploy/.pm2/logs/api-error.log` — API errors
- `/home/deploy/.pm2/logs/storefront-out.log` — Storefront stdout
- `/home/deploy/.pm2/logs/admin-out.log` — Admin stdout

Ngix Access & Error Logs

```
# Access logs (who is visiting)
tail -f /var/log/nginx/access.log

# Error logs (problems)
tail -f /var/log/nginx/error.log
```

Server Resource Monitoring

Install **htop** for quick server health checks:

```
sudo apt install -y htop
htop # interactive CPU/memory/process viewer
```

Check disk usage:

```
df -h # disk space
du -sh /home/deploy/naro-fashion # project size
```

Optional: PM2 Plus (free tier)

PM2 offers a web dashboard at <https://pm2.io> with free tier (1 server):

```
pm2 plus # follow prompts to link your server
```

Gives you: web-based monitoring, error tracking, deployment history, and alerts.

Part 7: Email Setup (Transactional Emails)

Your app needs email for: order confirmations, password resets, rental reminders, notifications.

Recommended: Brevo (formerly Sendinblue) — Free Tier

- **300 emails/day free** (9,000/month) — plenty for a new e-commerce site
- SMTP relay + API available
- Good deliverability

Setup Steps:

1. Sign up at <https://www.brevo.com> (free)
2. Go to **Settings** → **SMTP & API** → **SMTP**
3. Note your credentials:

- SMTP Server: `smtp-relay.brevo.com`
- Port: `587`
- Username: your Brevo email
- Password: generated SMTP key

4. Add to your production `.env` :

```
SMTP_HOST=smtp-relay.brevo.com
SMTP_PORT=587
SMTP_USER=your-brevo-email@example.com
SMTP_PASS=your-brevo-smtp-key
SMTP_FROM=noreply@narofashion.co.tz
```

5. **Verify your domain** in Brevo (Settings → Senders & Domains):

- Add `narofashion.co.tz`
- Add the required DNS records (SPF, DKIM, DMARC) to your domain registrar:

```
TXT    narofashion.co.tz      v=spf1 include:sendinblue.com ~all
TXT    mail._domainkey        (DKIM key from Brevo dashboard)
TXT    _dmarc.narofashion.co.tz v=DMARC1; p=quarantine; rua=mailto:your-email
```

Alternative Email Providers

Provider	Free Tier	Notes
Brevo	300/day	Recommended, good free tier
Resend	100/day (3,000/mo)	Developer-friendly API
Mailgun	100/day (first 3 months)	Then paid
Amazon SES	\$0.10/1000 emails	Very cheap, needs AWS account

Business Email (optional)

For receiving email at `info@narofashion.co.tz` :

- **Zoho Mail** free tier: 5 users, 5GB each — setup via Zoho admin panel
- Add MX records to your domain DNS as instructed by Zoho

Part 8: Files to Create/Modify

File	Action	Purpose
<code>.github/workflows/deploy.yml</code>	Create	CI/CD auto-deploy on push to master
<code>ecosystem.config.js</code> (root)	Create	PM2 process configuration
<code>deploy.sh</code> (root)	Create	Manual fallback deployment script
<code>apps/storefront/.env.local</code>	Modify	Production API URL

apps/admin/.env.local	Modify	Production API URL
.env (root)	Modify	Production secrets & URLs
packages/database/.env	Modify	Production DATABASE_URL

Part 9: Verification Checklist

After deployment, verify:

1. `https://narofashion.co.tz` — storefront loads, products display
2. `https://admin.narofashion.co.tz` — admin login works
3. `https://api.narofashion.co.tz/api/v1` — API responds
4. SSL certificate valid on all subdomains (check browser padlock)
5. `pm2 status` — all 3 processes running
6. Test a full flow: browse products → add to cart → checkout
7. Test admin flow: login → view dashboard → manage products

Cost Summary

Item	Cost
Hetzner CX32 VPS (4 vCPU, 8GB RAM)	~\$7.50/month
.co.tz domain (annual)	\$30/year (\$2.50/mo)
SSL (Let's Encrypt)	Free
GitHub Actions CI/CD	Free (2,000 min/mo)
UptimeRobot monitoring	Free
Brevo transactional email	Free (300/day)
Zoho business email (optional)	Free
Total	~\$10/month